

# SQL PLACEMENT PREPARATION NOTES

From absolute basics → simple queries → placement coding problems

**Goal:** These notes are designed for a beginner who wants to understand SQL first and then solve common campus-placement SQL questions. Do not memorize the queries blindly—understand what each clause does.

## PART 1 — SQL BASICS

### 1. What is SQL?

**SQL (Structured Query Language)** is a language used to communicate with relational databases. We use SQL to create tables, insert data, retrieve data, modify data, delete data, and control database objects.

**Example:** Asking the database to show all employees whose salary is greater than 50,000 is done using a SQL query.

### 2. DBMS vs RDBMS vs SQL

| Term  | Meaning                                                                                                              |
|-------|----------------------------------------------------------------------------------------------------------------------|
| DBMS  | Software used to create, store, manage and retrieve data from databases.                                             |
| RDBMS | A DBMS based on the relational model, where data is organized mainly in tables and relationships can be represented. |
| SQL   | Language used to interact with relational databases.                                                                 |

### 3. Database, Table, Row and Column

| Term           | Simple meaning                                     |
|----------------|----------------------------------------------------|
| Database       | A collection of related data managed by a DBMS.    |
| Table          | Data arranged in rows and columns.                 |
| Row / Record   | One complete entry in a table.                     |
| Column / Field | One attribute/property of the data.                |
| Cell           | One value at the intersection of a row and column. |

#### Example Employee table:

| emp_id | name  | department | salary |
|--------|-------|------------|--------|
| 101    | Anu   | IT         | 50000  |
| 102    | Ravi  | HR         | 45000  |
| 103    | Kiran | IT         | 65000  |

**Remember:** Table = collection of rows. Row = one record. Column = one type of information.

### 4. SQL Query Structure

Most beginner SQL queries follow this pattern:

```
SELECT columns FROM table WHERE condition;
```

**SELECT** → what you want. **FROM** → where the data comes from. **WHERE** → which rows should be selected.

### 5. SQL Statement Categories

| Category | Purpose                           | Examples                      |
|----------|-----------------------------------|-------------------------------|
| DDL      | Define/change database structure. | CREATE, ALTER, DROP, TRUNCATE |
| DML      | Insert/change/delete table data.  | INSERT, UPDATE, DELETE        |
| DQL      | Retrieve data.                    | SELECT                        |
| DCL      | Control permissions.              | GRANT, REVOKE                 |
| TCL      | Manage transactions.              | COMMIT, ROLLBACK, SAVEPOINT   |

■ Placement memory: **DDL** = structure, **DML** = data modification, **DQL** = query/retrieve, **DCL** = permissions, **TCL** = transactions.

## PART 2 — CREATING A TABLE & INSERTING DATA

### 6. CREATE TABLE

Creates the structure of a table.

```
CREATE TABLE Employee (  
    emp_id INT,  
    name VARCHAR(50),  
    department VARCHAR(30),  
    salary INT  
);
```

**INT** stores integer values. **VARCHAR(50)** stores variable-length text up to the specified limit.

### 7. INSERT

Adds rows to a table.

```
INSERT INTO Employee (emp_id, name, department, salary) VALUES (101, 'Anu', 'IT', 50000);
```

Multiple rows can be inserted depending on the DBMS syntax:

```
INSERT INTO Employee VALUES (102, 'Ravi', 'HR', 45000);
```

### 8. SELECT — Read Data

```
SELECT * FROM Employee;
```

\* means all columns.

To select specific columns:

```
SELECT name, salary FROM Employee;
```

### 9. WHERE — Filter Rows

```
SELECT name, salary FROM Employee WHERE salary > 50000;
```

Only rows satisfying the condition are returned.

### 10. Comparison Operators

| Operator | Meaning               | Example            |
|----------|-----------------------|--------------------|
| =        | Equal to              | salary = 50000     |
| <> or != | Not equal to          | department <> 'HR' |
| >        | Greater than          | salary > 50000     |
| <        | Less than             | salary < 50000     |
| >=       | Greater than or equal | salary >= 50000    |
| <=       | Less than or equal    | salary <= 50000    |

### 11. AND, OR, NOT

```
SELECT * FROM Employee WHERE department = 'IT' AND salary > 50000;
```

**AND**: both conditions must be true. **OR**: at least one condition must be true. **NOT**: reverses a condition.

### 12. UPDATE

```
UPDATE Employee SET salary = 55000 WHERE emp_id = 101;
```

**Important**: Always understand the WHERE condition. Without WHERE, an UPDATE can affect every row.

### 13. DELETE

```
DELETE FROM Employee WHERE emp_id = 101;
```

Without WHERE, DELETE can remove all rows from the table.

## PART 3 — IMPORTANT BASIC CLAUSES

### 14. ORDER BY

Sorts the result.

```
SELECT name, salary FROM Employee ORDER BY salary ASC;
```

ASC = ascending. DESC = descending.

### 15. DISTINCT

Removes duplicate values from the result.

```
SELECT DISTINCT department FROM Employee;
```

### 16. LIMIT

Restricts the number of returned rows in DBMSs that support LIMIT.

```
SELECT * FROM Employee ORDER BY salary DESC LIMIT 3;
```

### 17. LIKE

Used for pattern matching.

```
SELECT * FROM Employee WHERE name LIKE 'A%';
```

% = any number of characters. \_ = exactly one character.

### 18. BETWEEN

```
SELECT * FROM Employee WHERE salary BETWEEN 40000 AND 60000;
```

BETWEEN is inclusive in standard SQL usage: the boundary values are included.

### 19. IN

```
SELECT * FROM Employee WHERE department IN ('IT', 'HR');
```

Useful when checking whether a value matches one of several specified values.

### 20. IS NULL

NULL means missing/unknown value; it is not the same as 0 or an empty string.

```
SELECT * FROM Employee WHERE department IS NULL;
```

Do **not** normally write department = NULL. Use IS NULL or IS NOT NULL.

## PART 4 — AGGREGATE FUNCTIONS & GROUPING

### 21. COUNT

Counts rows or non-NULL values depending on the expression.

```
SELECT COUNT(*) FROM Employee;
```

### 22. SUM, AVG, MIN, MAX

```
SELECT SUM(salary), AVG(salary), MIN(salary), MAX(salary) FROM Employee;
```

| Function | Purpose            |
|----------|--------------------|
| COUNT()  | Count rows/values. |
| SUM()    | Total.             |
| AVG()    | Average.           |
| MIN()    | Smallest value.    |
| MAX()    | Largest value.     |

### 23. GROUP BY

GROUP BY creates groups of rows having the same value, so aggregate functions can be calculated for each group.

```
SELECT department, COUNT(*) FROM Employee GROUP BY department;
```

Result concept: one row per department with its employee count.

### 24. HAVING

**HAVING** filters groups after GROUP BY. **WHERE** filters individual rows before grouping.

```
SELECT department, COUNT(*) FROM Employee GROUP BY department HAVING COUNT(*) > 5;
```

■ Remember: **WHERE** → rows | **GROUP BY** → groups | **HAVING** → groups after grouping.

## PART 5 — JOINS: FIRST UNDERSTANDING

### 25. Why do we need JOIN?

Real databases often store related information in separate tables. A JOIN combines rows from tables using a related column.

Example tables: Employee(emp\_id, name, dept\_id) and Department(dept\_id, dept\_name).

### 26. INNER JOIN

Returns rows where the join condition matches in both tables.

```
SELECT e.name, d.dept_name FROM Employee e INNER JOIN Department d ON e.dept_id = d.dept_id;
```

### 27. LEFT JOIN

Returns every row from the left table and matching rows from the right table. If there is no match, right-table columns are NULL.

```
SELECT e.name, d.dept_name FROM Employee e LEFT JOIN Department d ON e.dept_id = d.dept_id;
```

### 28. RIGHT JOIN

Returns every row from the right table and matching rows from the left table, where supported.

### 29. FULL OUTER JOIN

Returns matching rows plus unmatched rows from both sides, where supported.

### 30. SELF JOIN

A table is joined with itself. This is especially useful for employee-manager relationships.

```
SELECT e.name AS employee, m.name AS manager FROM Employee e JOIN Employee m ON e.manager_id = m.emp_id;
```

■ Placement memory: **INNER** = matching; **LEFT** = all left + matches; **RIGHT** = all right + matches; **SELF** = same table joined to itself.

## PART 6 — SUBQUERIES & CASE

### 31. Subquery

A **subquery** is a query written inside another query. The inner query usually produces a value or set that the outer query uses.

```
SELECT name, salary FROM Employee WHERE salary > (SELECT AVG(salary) FROM Employee);
```

Here the inner query calculates the average salary. The outer query finds employees above that average.

### 32. EXISTS

**EXISTS** checks whether the subquery returns at least one row.

```
SELECT d.dept_name FROM Department d WHERE EXISTS (SELECT 1 FROM Employee e WHERE e.dept_id = d.dept_id);
```

### 33. CASE

CASE works like conditional logic and can create a value based on conditions.

```
SELECT name, salary, CASE WHEN salary >= 60000 THEN 'High' ELSE 'Normal' END AS category FROM Employee;
```

### 34. UNION vs UNION ALL

| UNION                                                      | UNION ALL                                  |
|------------------------------------------------------------|--------------------------------------------|
| Combines compatible result sets and removes duplicate rows | Combines result sets and keeps duplicates. |
| May require duplicate-elimination work.                    | Usually avoids duplicate elimination.      |

## PART 7 — WINDOW FUNCTIONS FOR PLACEMENTS

### 35. ROW\_NUMBER, RANK, DENSE\_RANK

Window functions calculate values across related rows without collapsing the rows into one group.

```
SELECT name, salary, ROW_NUMBER() OVER (ORDER BY salary DESC) AS rn FROM Employee;
```

**ROW\_NUMBER**: gives a unique sequence. **RANK**: same values receive the same rank and gaps can appear.

**DENSE\_RANK**: same values receive the same rank but no gaps appear.

| Salary | ROW_NUMBER | RANK | DENSE_RANK |
|--------|------------|------|------------|
| 90000  | 1          | 1    | 1          |
| 80000  | 2          | 2    | 2          |
| 80000  | 3          | 2    | 2          |
| 70000  | 4          | 4    | 3          |

## PART 8 — SQL CODING PROBLEMS: YOUR 25 QUESTIONS

**Important:** These are practice problems. First understand the basic clauses above. Then use these as your placement problem set. Queries below use common SQL syntax; minor syntax differences can exist across MySQL, PostgreSQL, Oracle, and SQL Server.

**Sample Employee table used for many problems:**

| emp_id | name  | department | salary | manager_id | join_date  |
|--------|-------|------------|--------|------------|------------|
| 1      | Asha  | IT         | 90000  | NULL       | 2024-01-10 |
| 2      | Ravi  | IT         | 80000  | 1          | 2024-06-15 |
| 3      | Kiran | HR         | 70000  | NULL       | 2023-08-12 |
| 4      | Meena | HR         | 70000  | 3          | 2024-02-20 |

| emp_id | name | department | salary | manager_id | join_date  |
|--------|------|------------|--------|------------|------------|
| 5      | John | IT         | 60000  | 1          | 2025-01-05 |

## 1. Find Second Highest Salary

```
SELECT MAX(salary) AS second_highest FROM Employee WHERE salary < (SELECT MAX(salary) FROM Employee);
```

**Explanation:** Inner MAX finds the highest salary. WHERE removes that salary. Outer MAX finds the largest remaining salary. This is a classic subquery problem.

## 2. Find Third Highest Salary

```
SELECT MAX(salary) AS third_highest FROM Employee WHERE salary < (SELECT MAX(salary) FROM Employee WHERE salary < (SELECT MAX(salary) FROM Employee));
```

**Explanation:** The nested conditions remove the first and second highest distinct salary levels, then MAX returns the third.

## 3. Find Nth Highest Salary

For modern SQL, a common approach uses DENSE\_RANK:

```
SELECT salary FROM (SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rn FROM Employee) x WHERE rn = 3;
```

**Explanation:** DENSE\_RANK assigns 1 to the highest distinct salary, 2 to the next, and so on. Change 3 to N.

## 4. Find Duplicate Records

```
SELECT name, department, salary, COUNT(*) AS cnt FROM Employee GROUP BY name, department, salary HAVING COUNT(*) > 1;
```

**Explanation:** GROUP BY makes identical combinations one group. HAVING COUNT(\*) > 1 keeps groups appearing more than once.

## 5. Remove Duplicate Records

A common approach is to identify duplicates with ROW\_NUMBER and delete rows with rn > 1; exact DELETE syntax differs by DBMS. Conceptually:

```
ROW_NUMBER() OVER (PARTITION BY duplicate_columns ORDER BY emp_id)
```

**Explanation:** Keep rn = 1 and remove later copies. Always test the SELECT version before running DELETE.

## 6. Employees Earning Above Average

```
SELECT * FROM Employee WHERE salary > (SELECT AVG(salary) FROM Employee);
```

**Explanation:** Inner AVG calculates the overall average. Outer WHERE compares each employee against it.

## 7. Employees Earning Maximum Salary

```
SELECT * FROM Employee WHERE salary = (SELECT MAX(salary) FROM Employee);
```

**Explanation:** MAX gets the highest salary; equality returns every employee tied at that salary.

## 8. Maximum Salary Department-wise

```
SELECT department, MAX(salary) AS max_salary FROM Employee GROUP BY department;
```

**Explanation:** GROUP BY creates one group per department; MAX is calculated inside each group.

## 9. Second Highest Salary Department-wise

```
SELECT department, salary FROM (SELECT department, salary, DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rn FROM Employee) x WHERE rn = 2;
```

**Explanation:** PARTITION BY restarts ranking for every department. rn = 2 means the second distinct salary in that department.

## 10. Count Employees Department-wise

```
SELECT department, COUNT(*) AS employee_count FROM Employee GROUP BY department;
```

**Explanation:** GROUP BY department creates department groups; COUNT counts rows in each group.

## 11. Departments With More Than 5 Employees

```
SELECT department, COUNT(*) AS employee_count FROM Employee GROUP BY department HAVING COUNT(*) > 5;
```

**Explanation:** Because COUNT is an aggregate, its group-level condition belongs in HAVING.

## 12. Employees Without Departments

```
SELECT * FROM Employee WHERE department IS NULL;
```

**Explanation:** NULL must be checked using IS NULL, not = NULL.

## 13. Departments Without Employees

```
SELECT d.* FROM Department d LEFT JOIN Employee e ON d.dept_id = e.dept_id WHERE e.emp_id IS NULL;
```

**Explanation:** LEFT JOIN keeps every department. Departments with no matching employee have NULL employee columns.

## 14. Employees Whose Salary Is Greater Than Manager

```
SELECT e.name AS employee, e.salary, m.name AS manager, m.salary AS manager_salary FROM Employee e JOIN Employee m ON e.manager_id = m.emp_id WHERE e.salary > m.salary;
```

**Explanation:** This is a SELF JOIN. e represents employee and m represents manager.

## 15. Employees Who Joined in a Particular Year

Common MySQL-style example:

```
SELECT * FROM Employee WHERE YEAR(join_date) = 2024;
```

**Explanation:** YEAR extracts the year. Date-function syntax varies by DBMS.

## 16. Find Duplicate Salaries

```
SELECT salary, COUNT(*) AS cnt FROM Employee GROUP BY salary HAVING COUNT(*) > 1;
```

**Explanation:** Group by salary and keep salary groups occurring more than once.

## 17. Find Top 3 Salaries

```
SELECT DISTINCT salary FROM Employee ORDER BY salary DESC LIMIT 3;
```

**Explanation:** DISTINCT treats equal salaries as one salary level. ORDER BY DESC puts highest first; LIMIT keeps three.

## 18. Top 3 Salaries Department-wise

```
SELECT department, name, salary FROM (SELECT department, name, salary, DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rn FROM Employee) x WHERE rn <= 3;
```

**Explanation:** Ranking restarts for each department. rn <= 3 keeps the top three distinct salary levels per department.

## 19. Find Consecutive Records

A common approach uses LAG/LEAD or row-number differences. Example for consecutive employee IDs:

```
SELECT emp_id, LAG(emp_id) OVER (ORDER BY emp_id) AS previous_id FROM Employee;
```

**Explanation:** LAG gives the previous row's ID. Compare current and previous values to detect consecutive records. The exact definition of 'consecutive' depends on the problem.

## 20. Customers Who Never Ordered

```
SELECT c.* FROM Customers c LEFT JOIN Orders o ON c.customer_id = o.customer_id WHERE o.order_id IS NULL;
```

**Explanation:** LEFT JOIN keeps all customers; NULL order\_id identifies customers with no matching order.

## 21. Customers Who Ordered More Than Once

```
SELECT customer_id, COUNT(*) AS order_count FROM Orders GROUP BY customer_id HAVING COUNT(*) > 1;
```

**Explanation:** Group orders by customer and keep customers whose count is greater than one.

## 22. Products Never Purchased

```
SELECT p.* FROM Products p LEFT JOIN OrderItems oi ON p.product_id = oi.product_id WHERE oi.product_id IS NULL;
```

**Explanation:** LEFT JOIN preserves every product; unmatched products have NULL order-item data.

## 23. Highest Order Amount

```
SELECT MAX(order_amount) AS highest_order_amount FROM Orders;
```

**Explanation:** MAX returns the largest order amount.

## 24. Average Salary Department-wise

```
SELECT department, AVG(salary) AS avg_salary FROM Employee GROUP BY department;
```

**Explanation:** GROUP BY creates one group per department and AVG calculates each group's average.

## 25. Employees Sharing the Same Salary

```
SELECT * FROM Employee WHERE salary IN (SELECT salary FROM Employee GROUP BY salary HAVING COUNT(*) > 1);
```

**Explanation:** The inner query finds salaries appearing multiple times. The outer query returns all employees having one of those salaries.

# FINAL — WHAT TO MEMORIZE FOR PLACEMENTS

| Concept             | Memory shortcut                            |
|---------------------|--------------------------------------------|
| SELECT              | Choose columns.                            |
| FROM                | Choose table/source.                       |
| WHERE               | Filter rows.                               |
| ORDER BY            | Sort result.                               |
| DISTINCT            | Remove duplicate result values.            |
| GROUP BY            | Create groups.                             |
| HAVING              | Filter groups.                             |
| JOIN                | Combine related tables.                    |
| LEFT JOIN + IS NULL | Find records with no match.                |
| Subquery            | Query inside another query.                |
| MAX/MIN/AVG/COUNT   | Common aggregate functions.                |
| DENSE_RANK          | Useful for Nth/top salaries with ties.     |
| PARTITION BY        | Restart window calculation for each group. |
| SELF JOIN           | Compare rows within the same table.        |
| CASE                | Conditional logic inside SQL.              |

**Recommended learning order:** Do not start by memorizing the 25 answers. First practice SELECT → WHERE → ORDER BY → GROUP BY → HAVING → JOIN → subqueries → window functions. Then solve the 25 placement problems without looking at the answers.